

SMARTLOGIX

DESARROLLO FULLSTACK III_003D



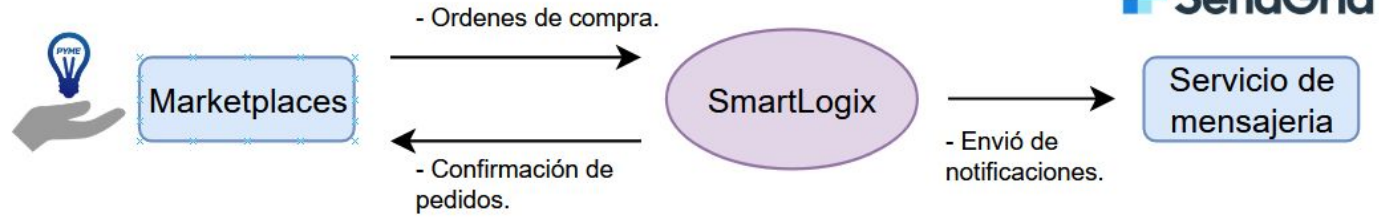
Integrantes:
Isak Chacana
Gabriel Jeria
Vicente Palacios

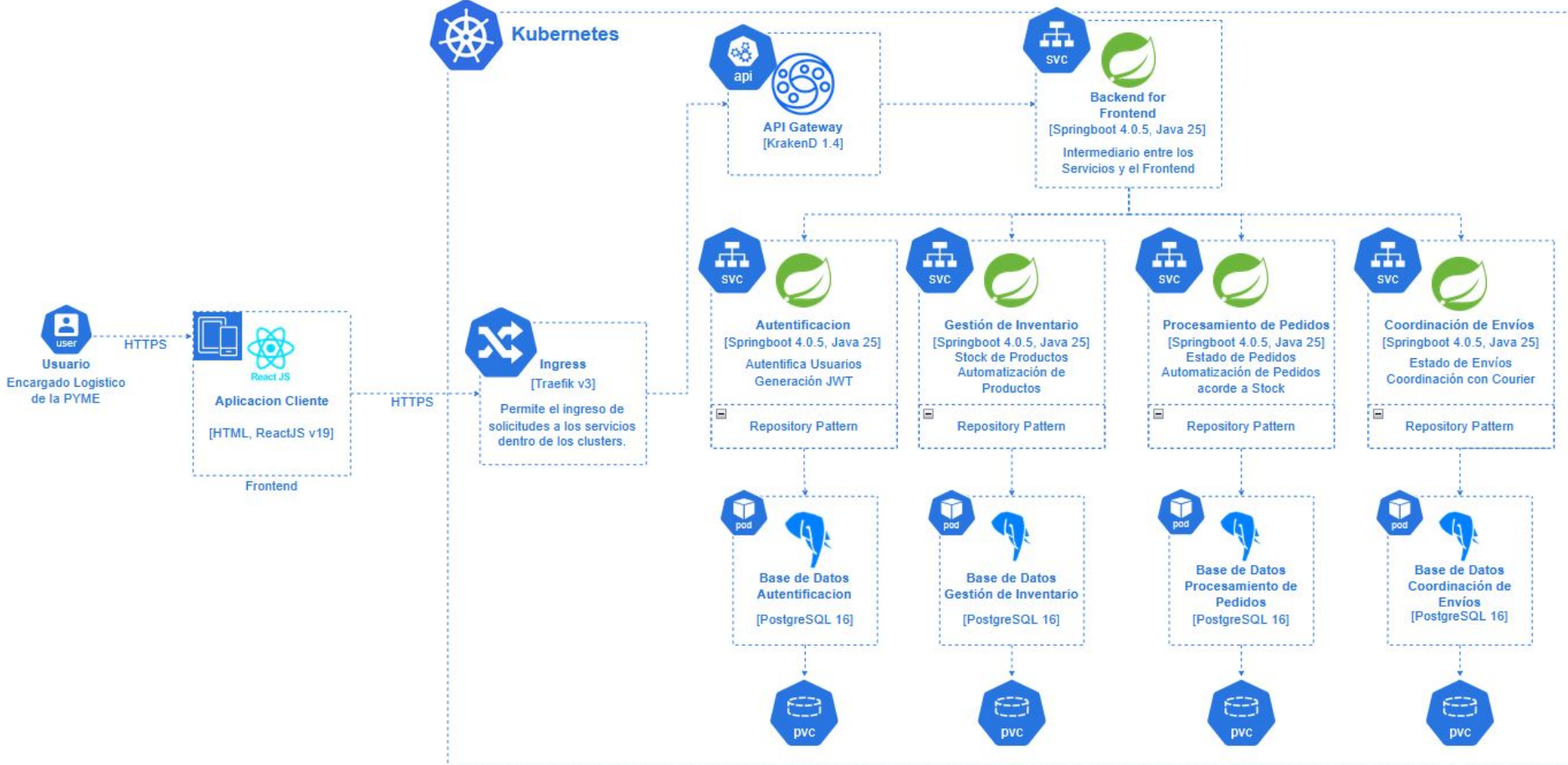
Docente:
Claudio Esteban Rojas Rodriguez

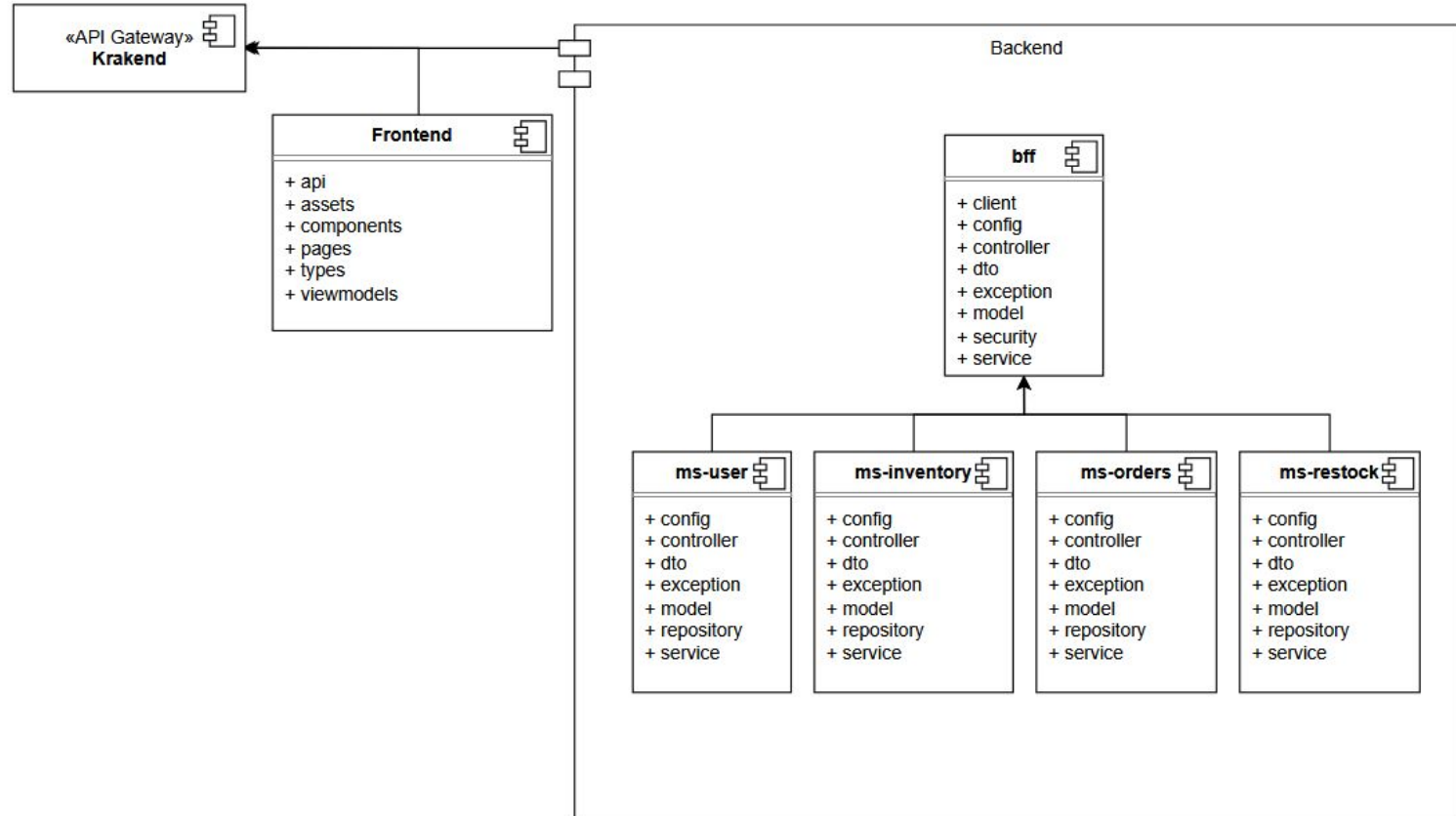
Introduccion.

SmartLogix es una startup que busca optimizar la gestión logística de pequeñas y medianas empresas mediante una plataforma basada en microservicios. La solución permite automatizar procesos de inventario, pedidos y envíos, reduciendo errores, mejorando la trazabilidad y facilitando la escalabilidad de las operaciones logísticas.

En esta tercera entrega se implementaron los microservicios de Orders y Restock, pruebas unitarias, kubernetes y refactorización del proyecto.







Patrones.

Arquitectónico.

BFF.

Centraliza las peticiones del frontend y redirige hacia los microservicios.

MVVM.

Las vistas son las páginas, el ViewModel maneja el estado y las APIs conectan con el backend.

MVC.

Controller recibe la petición, Service aplica la lógica y Model representa los datos del sistema.

Diseño.

DTO Pattern.

Transporta datos entre capas sin exponer directamente las entidades.

Repository Pattern.

Separa el acceso a datos de la lógica de negocio.

Proxy Pattern.

El BFF usa clientes como `UserServiceClient` o `RestockServiceClient` para comunicarse con otros microservicios.

Pruebas Unitarias

Las pruebas unitarias fueron implementadas con Junit 5 y Mockito sobre capas Service y Controller de cada microservicio. Mockito permite aislar la lógica de negocio simulando las dependencias (repositorios y servicios) sin requerir base de datos real.

la cobertura fue medida con JaCoCo, obteniendo los siguientes resultados.

Pruebas Unitarias

Microservicio	Cobertura
ms-user	100%
ms-inventory	100%
ms-orders	100%
ms-restock	80%

Todos los microservicios superan el mínimo exigido del 60% de cobertura, validando la correcta implementación de la lógica de negocio en cada componente del sistema.

Microservicio Orders.

service-orders es un microservicio REST independiente construido con Spring Boot. Gestiona la creación, consulta y seguimiento de órdenes de compra en SmartLogix. Además, se integra con Inventory para validar y actualizar el stock de los productos solicitados.

Endpoints Principales:

GET /api/orders → lista órdenes
POST /api/orders → crea orden
GET /api/orders/{id} → obtiene orden
PUT /api/orders/{id}/status → actualiza estado
DELETE /api/orders/{id} → elimina orden

Stack Técnico:

Al ser un microservicio independiente, puede desplegarse y escalarse sin afectar a los demás servicios. Utiliza PostgreSQL para persistencia de datos y se comunica con Inventory mediante Feign Client para validar y descontar stock. Todas las solicitudes pasan a través del BFF.

Microservicio Re-Stock.

service-restock es un microservicio REST independiente construido con Spring Boot. Gestiona las solicitudes de reposición de stock de SmartLogix, permitiendo registrar, aprobar y dar seguimiento a los requerimientos de abastecimiento de productos almacenados en las distintas bodegas.

Endpoints Principales:

GET /api/restock → lista solicitudes

POST /api/restock → crea solicitud

GET /api/restock/{id} → obtiene solicitud

PUT /api/restock/{id}/estado → actualiza estado

DELETE /api/restock/{id} → elimina solicitud

Stack Técnico:

Restock utiliza Spring Boot, PostgreSQL y OpenFeign. Se integra con Inventory para identificar los productos que requieren reposición y mantener consistencia en la gestión de stock. Al ser un microservicio independiente, puede escalarse o actualizarse sin afectar el resto de la plataforma.

Conclusion

En conclusión, SmartLogix propone una solución moderna basada en microservicios para optimizar la gestión logística de las PYMEs. Durante esta tercera etapa se implementaron los microservicios de orders y restock, aplicando tecnologías y patrones de diseño que permiten una arquitectura escalable, desacoplada y preparada para futuras integraciones y automatizaciones.



smartlogix